

Architecture de la Solution

L'objectif de ce projet est de concevoir une architecture capable de supporter les différents besoins fonctionnels d'une application bancaire tout en garantissant une organisation claire, évolutive et maintenable.

Réalisé par **Dylan BAUDSON, Guillaume CROUZET et Grégory WOLFF**

ARCHITECTURE LOGICIELLE

MICROSERVICES

HEXAGONALE

BanqueApp

Gestion bancaire

C Clients

A Comptes

T Transactions

Déconnexion

Clients

CRÉER UN CLIENT

Nom

Dupont

Prénom

Jean

Email

jean@dupont.fr

Créer

```
{
  "id": "client_6a4a24a09ceba",
  "nom": "Dupont",
  "prenom": "Jean",
  "email": "jean@dupont.com"
}
```

LISTE DES CLIENTS

ID CLIENT

client_6a4a24a09ceba

client_6a4a27cc2661c

client_6a4a4eb9e9b63

1. Contexte

Avant toute implémentation, une analyse fonctionnelle a permis d'identifier les principaux domaines métier de l'application bancaire. Cette réflexion a mis en évidence des responsabilités distinctes qui, bien qu'elles collaborent entre elles, possèdent leurs propres règles et évoluent indépendamment.

Authentification

Gestion de l'identité des utilisateurs et génération des jetons JWT.

Clients

Administration des informations relatives aux clients de la banque.

Comptes Bancaires

Gestion des soldes et des opérations sur les comptes.

Transactions

Réalisation des virements et consultation de l'historique.

Ces domaines, bien qu'ils collaborent entre eux, possèdent leurs propres règles métier et évoluent indépendamment. L'architecture a donc été pensée pour refléter cette séparation fonctionnelle.

L'objectif n'était pas uniquement de répartir le code dans plusieurs projets, mais surtout d'obtenir une architecture où chaque composant possède une responsabilité clairement identifiée.

2. Principes de Conception

Quatre principes directeurs ont guidé l'ensemble des choix architecturaux du projet, formant le socle de cohérence de la solution.

Responsabilité Unique

Chaque service ne traite qu'un seul domaine fonctionnel, limitant ainsi les dépendances entre les parties du système et facilitant l'évolution indépendante de chaque composant.

Séparation Métier / Technique

Les règles métier sont totalement indépendantes du framework, du protocole HTTP et du système de stockage. Le domaine reste pur et testable sans infrastructure.

Homogénéité

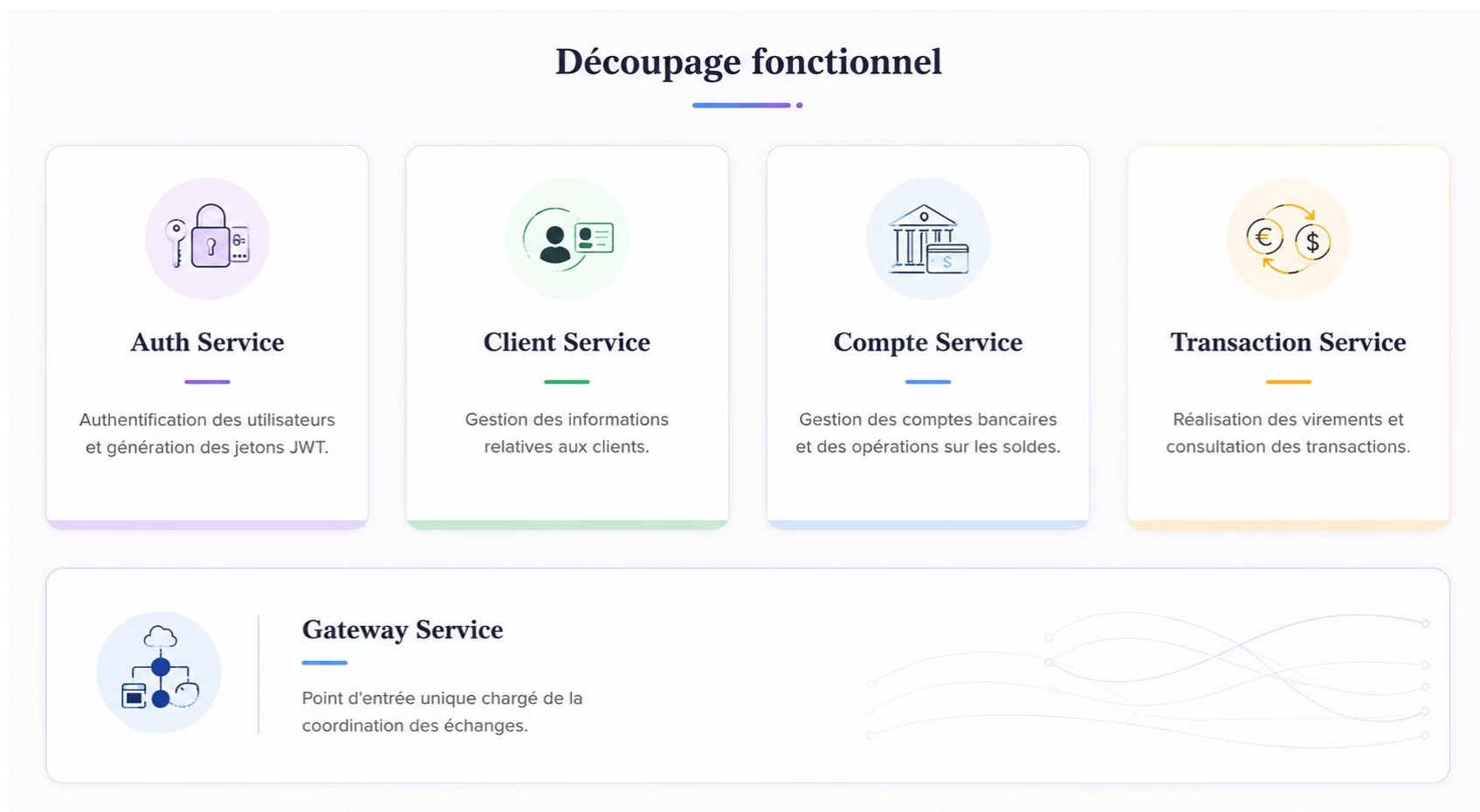
Tous les microservices suivent la même structure organisationnelle, ce qui simplifie la compréhension, la maintenance et le développement de nouvelles fonctionnalités.

Centralisation Transverse

Les fonctionnalités transverses — authentification, journalisation, CORS — sont centralisées dans un composant dédié pour éviter toute duplication entre les services.

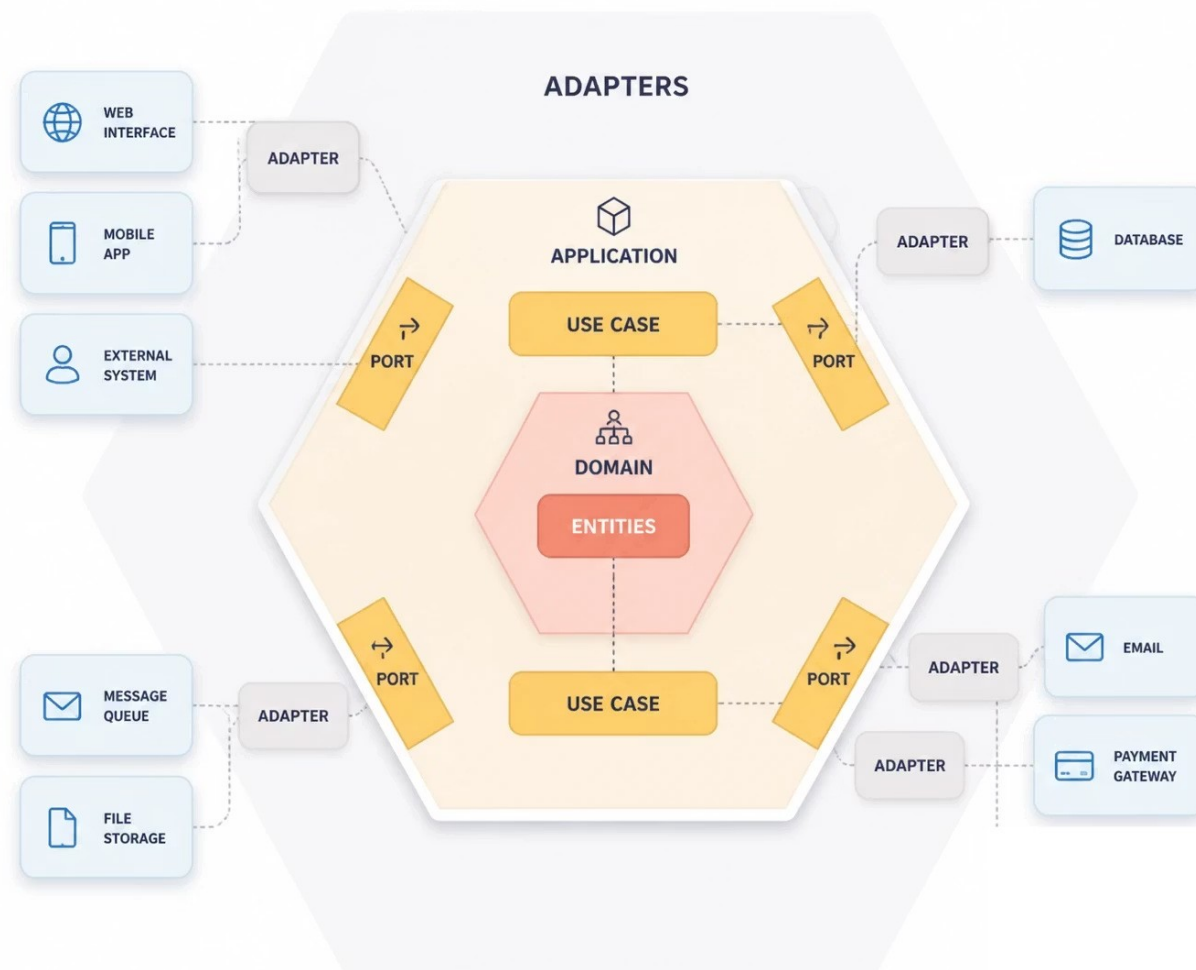
3. Découpage Fonctionnel en Microservices

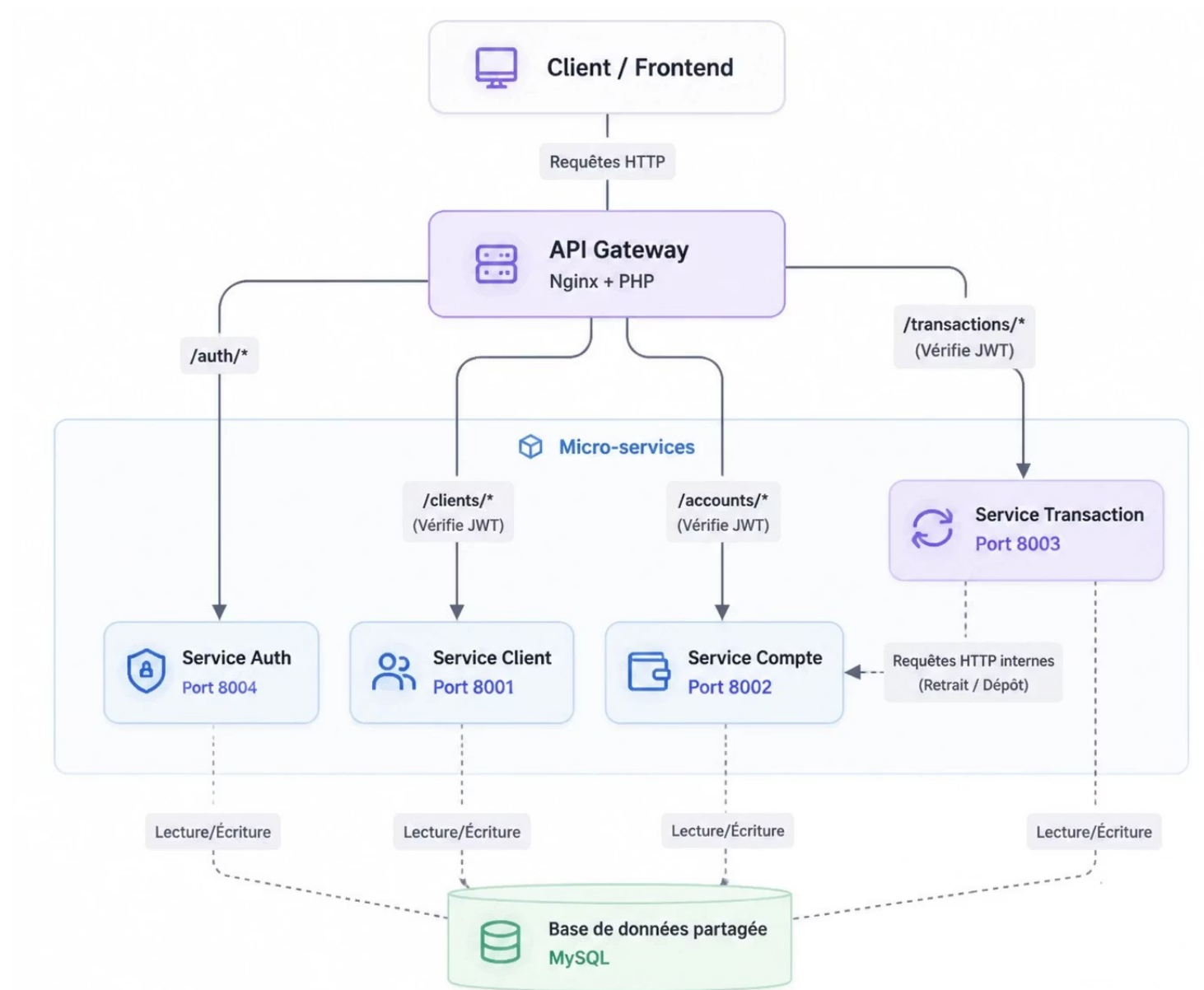
Le découpage fonctionnel définit cinq services autonomes, chacun responsable d'un domaine métier précis. Cette organisation permet de faire évoluer un domaine sans impacter les autres composants du système.



Vue d'Ensemble de l'Architecture

Le découpage fonctionnel présenté précédemment définit les responsabilités de chaque domaine métier. Il est désormais nécessaire de comprendre comment ces différents services collaborent au sein de l'application et comment les échanges sont organisés.





4. Architecture des services

Une fois les domaines identifiés, une seconde problématique est apparue : comment permettre aux clients d'accéder simplement aux différents services ?

Le choix retenu est l'utilisation d'un **API Gateway** comme unique point d'entrée.

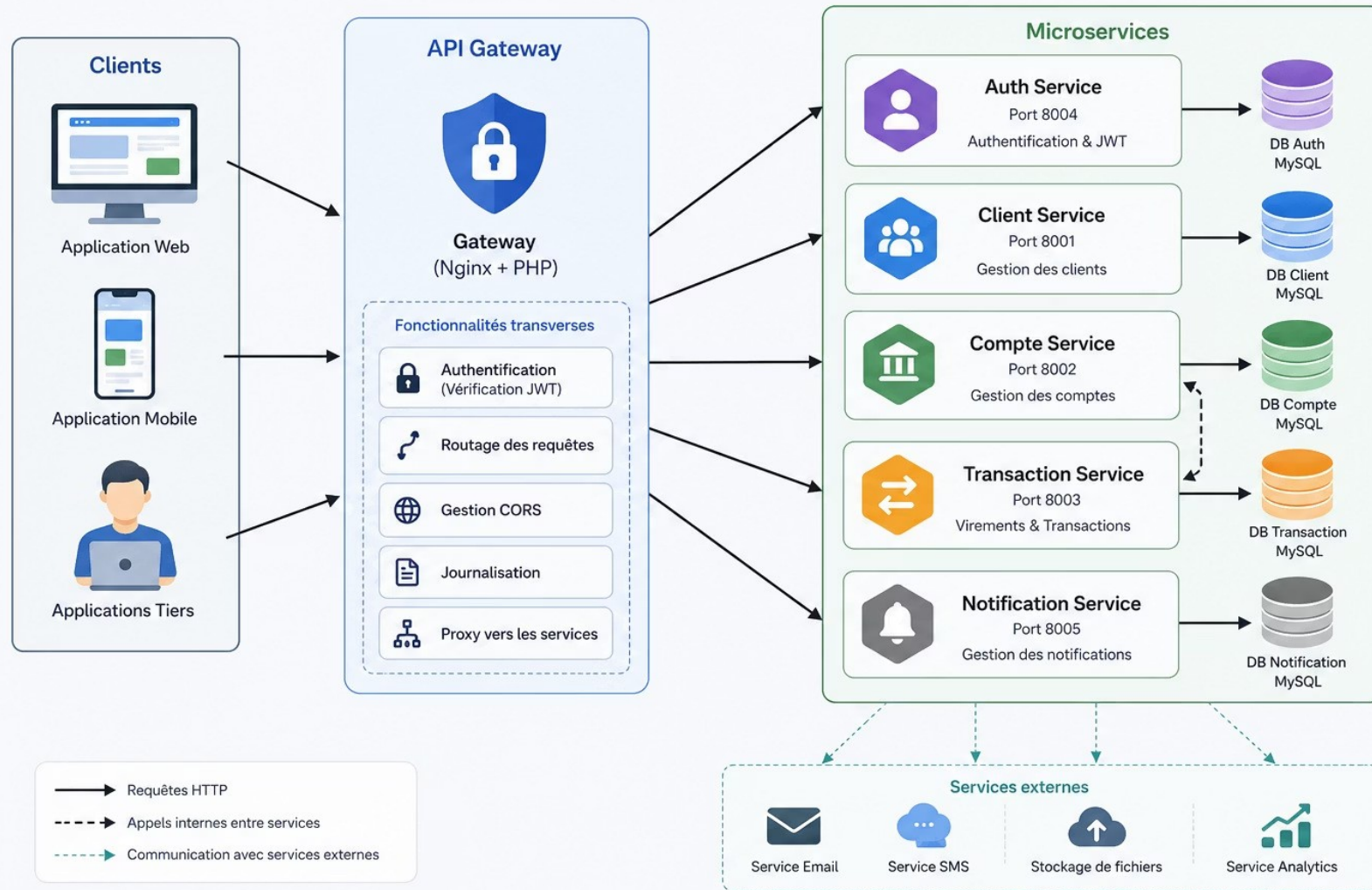
Toutes les requêtes transitent par ce composant avant d'être redirigées vers le microservice concerné.

Cette approche permet de centraliser plusieurs traitements techniques communs :

le routage des requêtes ; la validation des jetons JWT ; la gestion du CORS ; la journalisation des requêtes ; le proxy HTTP vers les différents services.

Architecture de la Solution

Application Bancaire - Architecture Microservices



Les microservices restent ainsi concentrés exclusivement sur leur logique métier.

Le Gateway ne contient aucune règle fonctionnelle ; il joue uniquement un rôle de coordination entre le client et les services.

5. Architecture interne des microservices

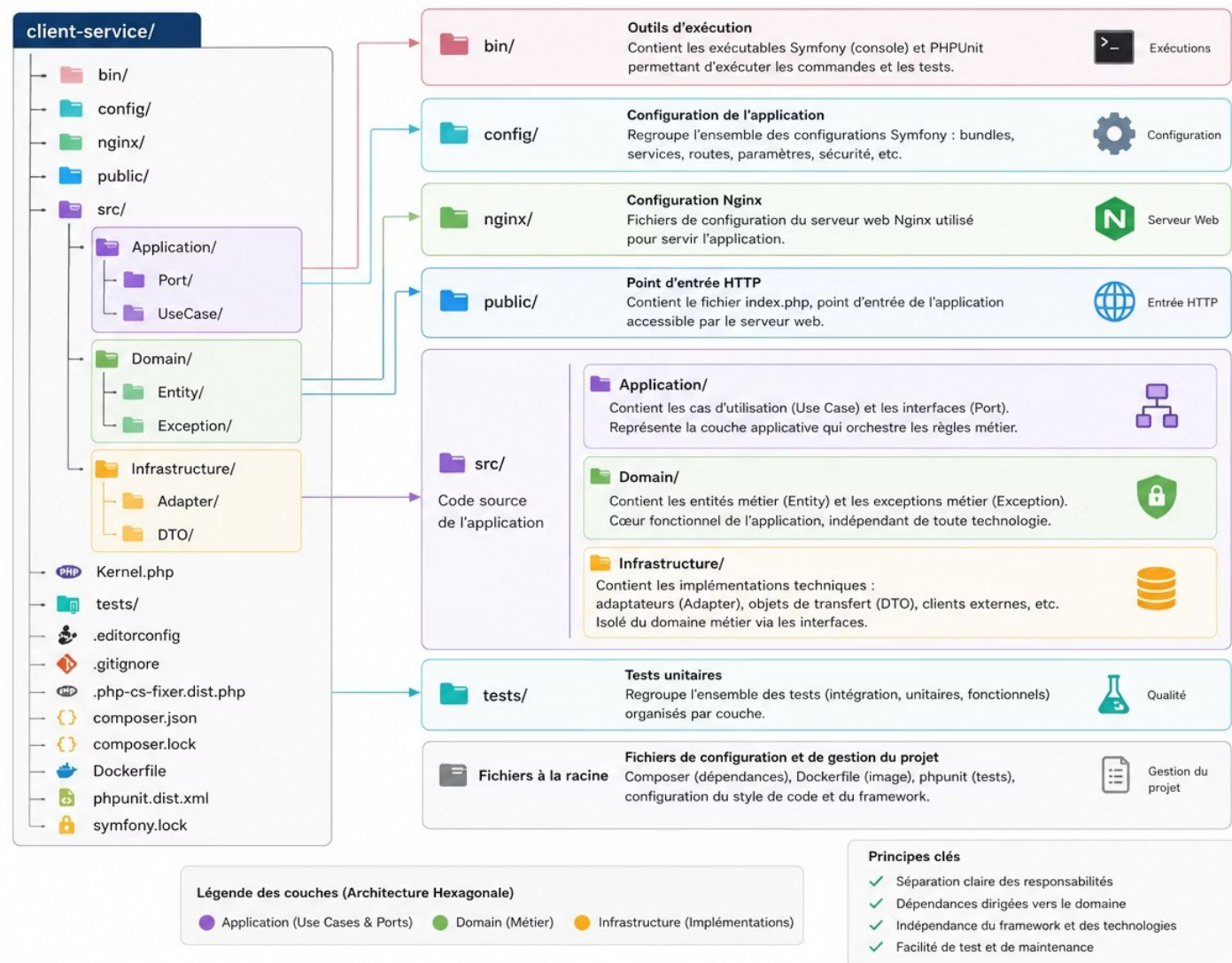
Une fois l'architecture globale définie, une réflexion a été menée sur l'organisation interne de chaque microservice.

L'objectif était de construire des services indépendants des technologies employées et de préserver les règles métier des préoccupations techniques.

La figure suivante présente l'organisation interne retenue. Elle illustre la séparation entre les différentes couches composant un microservice ainsi que les dépendances autorisées entre celles-ci

STRUCTURE INTERNE D'UN MICRO-SERVICE

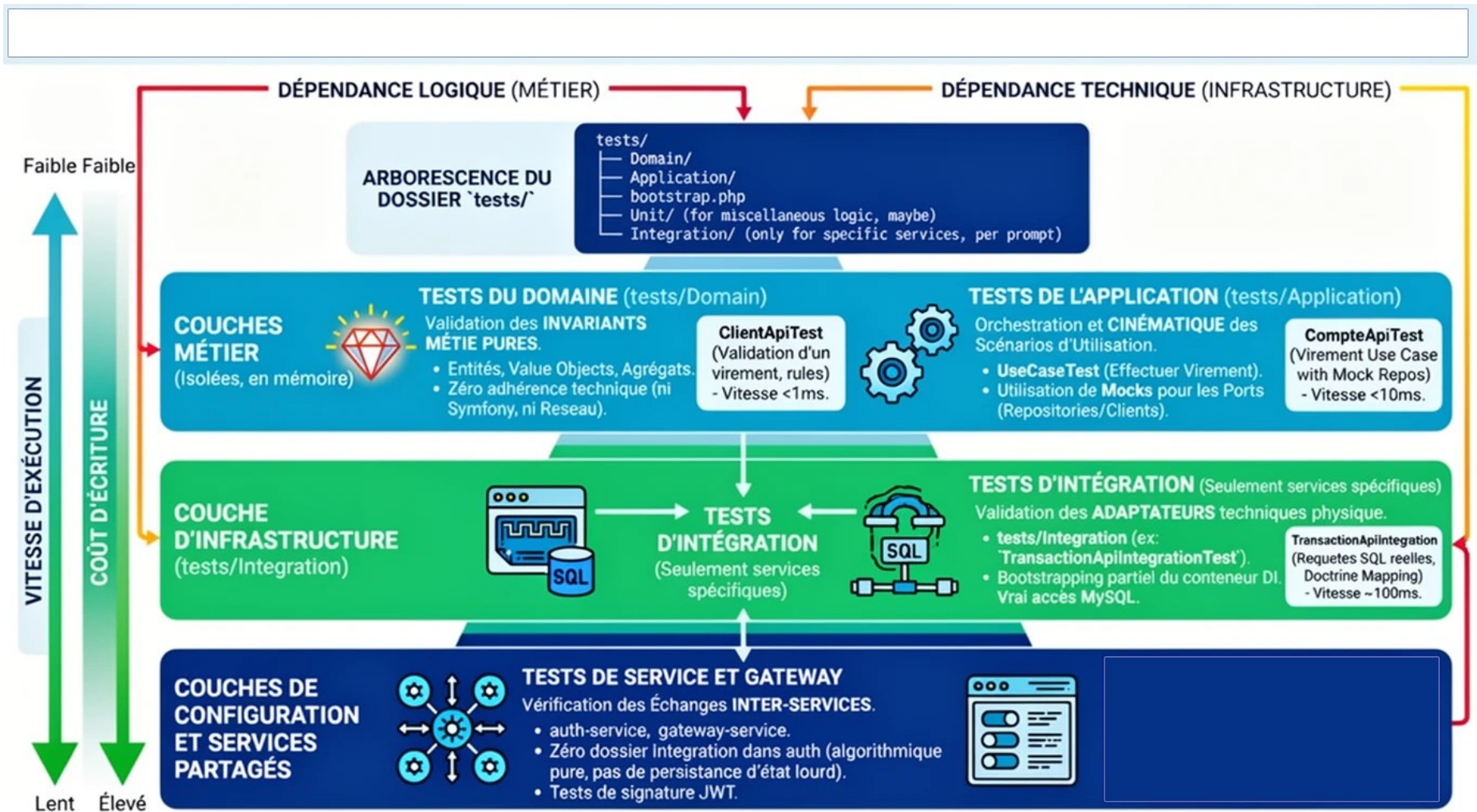
Exemple : client-service



Le contrôleur HTTP constitue le point d'entrée du microservice. Il délègue immédiatement le traitement au cas d'utilisation correspondant, situé dans la couche **Application**. Ce dernier orchestre les opérations métier en manipulant les entités du domaine et en s'appuyant uniquement sur des interfaces (Ports). Les implémentations concrètes, telles que les repositories ou les clients HTTP, sont regroupées dans la couche **Infrastructure**. Cette organisation garantit que le domaine métier reste indépendant des technologies employées, conformément aux principes de l'architecture hexagonale.

6. Stratégie de Test & Isolation des Couches

Une bonne architecture logicielle doit être facile à tester. C'est pourquoi nous avons découpé nos tests en trois niveaux distincts, chacun ayant pour rôle de valider une couche précise de nos microservices sans interférer avec les autres.



Tests de l'Application (tests/Application)

Ils vérifient le bon déroulement de nos scénarios d'utilisation (Use Cases). Pour rester isolés de la technique, on utilise des doublons (Mocks) qui simulent les accès à la base de données ou aux autres services.

Tests d'Intégration (tests/Integration)

Ils valident toute la partie technique (Infrastructure). C'est ici que l'on test si nos requêtes SQL (Doctrine) fonctionnent et si nos microservices communiquent correctement entre eux.

Une structure en miroir

Le dossier tests/reprend exactement le découpage de src/(Domain, Application, Integration). Cela prouve que notre code de test respecte les mêmes règles d'isolation que notre code de production.

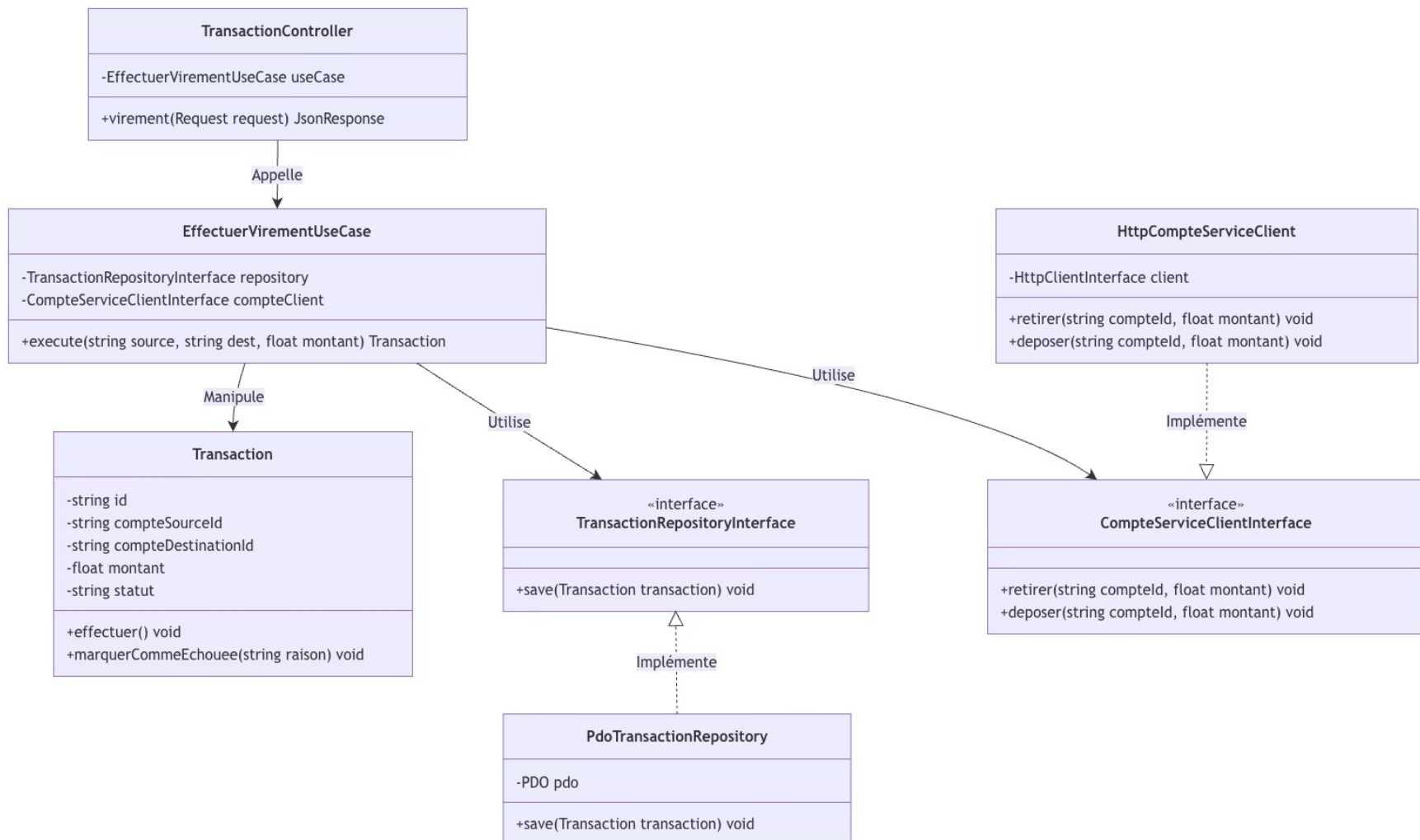
Tests du Domaine

Ils testent uniquement la logique métier pure (comme les règles de calcul ou la validation des données d'un virement).

Ils s'exécutent en mémoire, ultra-rapidement, sans jamais toucher à la base de données ou au framework Symfony.

7. Modèle métier

L'architecture présentée jusqu'à présent décrit l'organisation technique de l'application. Il convient désormais de présenter les principales entités métier manipulées par le système bancaire ainsi que les relations qui les unissent.



Le modèle métier repose sur quatre entités principales : **Utilisateur**, **Client**, **Compte** et **Transaction**.

Un utilisateur correspond à une identité permettant l'authentification au sein de l'application.

Un client représente une personne physique pouvant posséder un ou plusieurs comptes bancaires.

Chaque compte est associé à son propriétaire et contient les informations relatives à son solde.

Enfin, une transaction matérialise un mouvement financier entre un compte source et un compte destinataire, tout en conservant les informations nécessaires au suivi de son exécution, telles que le montant, la date et son statut.

Ce modèle constitue le socle des règles métier implémentées dans les différents microservices.

Utilisateur

Identité permettant l'authentification au sein de l'application.

Client

Personne physique pouvant posséder un ou plusieurs comptes bancaires.

Compte

Associé à son propriétaire, contient les informations relatives au solde.

Transaction

Mouvement financier entre un compte source et un compte destinataire, avec montant, date et statut.

8. Cas particulier du Gateway

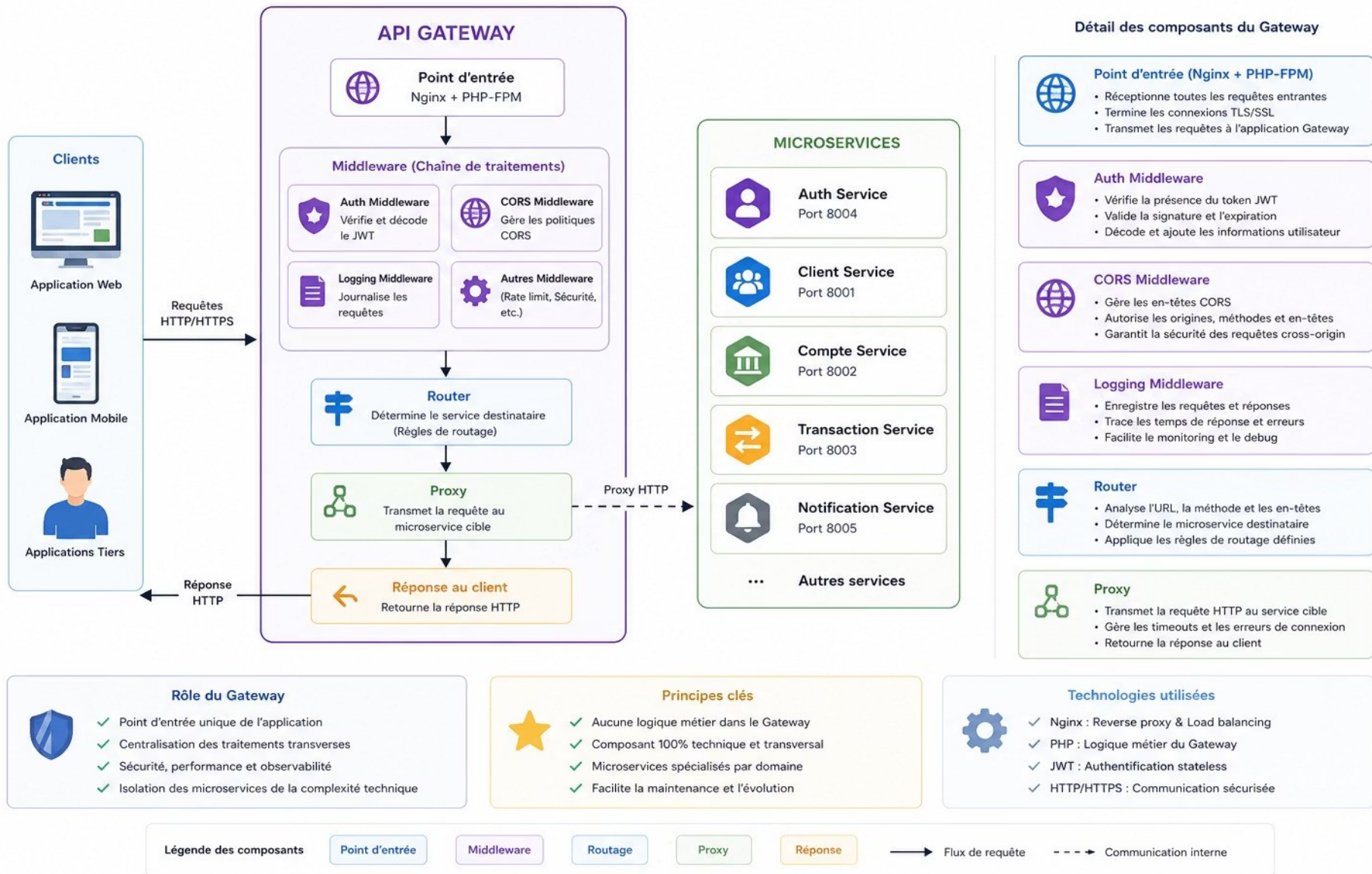
Contrairement aux autres microservices, le Gateway ne contient aucune logique métier. Son organisation est donc différente :

- **Router** : détermine le service destinataire.
- **Proxy** : transmet les requêtes HTTP.
- **Middleware** : applique les traitements transverses (JWT, CORS, logs).
- **Auth** : vérifie et décode les jetons JWT.

Cette architecture permet de centraliser tous les traitements communs dans un seul composant. Les microservices restent ainsi

STRUCTURE INTERNE DU GATEWAY

Composant transversal – Rôle exclusivement technique



9. Flux de traitement d'une requête



Justification des Choix Architecturaux

→ Découpage par domaines métier

Isole les responsabilités et limite les dépendances entre services.

→ Architecture hexagonale

Garantit l'indépendance du domaine métier, améliore la testabilité et facilite les évolutions.

→ API Gateway centralisé

Simplifie les échanges clients et centralise authentification, logs et CORS.

→ Homogénéité des services

Chaque nouveau service reprend la même organisation, réduisant le temps de prise en main et simplifiant la maintenance.



L'ensemble de ces choix aboutit à une architecture **modulaire, lisible et évolutive**, dans laquelle chaque composant possède une responsabilité clairement définie.